

Chuẩn bị buổi 6 Demo-Day

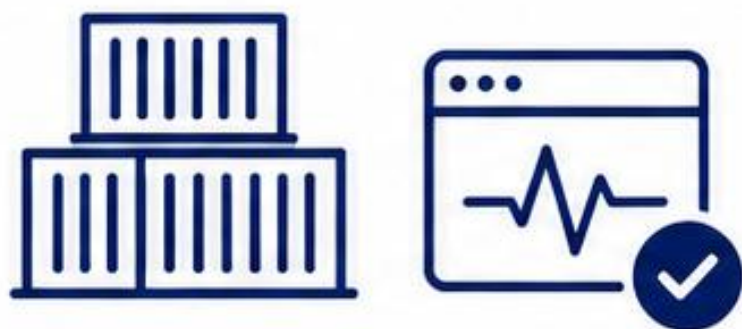
Giảng viên: Ths. Lê Thái Bảo

DAI NAM
UNIVERSITY

BUỔI 6 KIỂM TRA ĐIỀU GÌ?

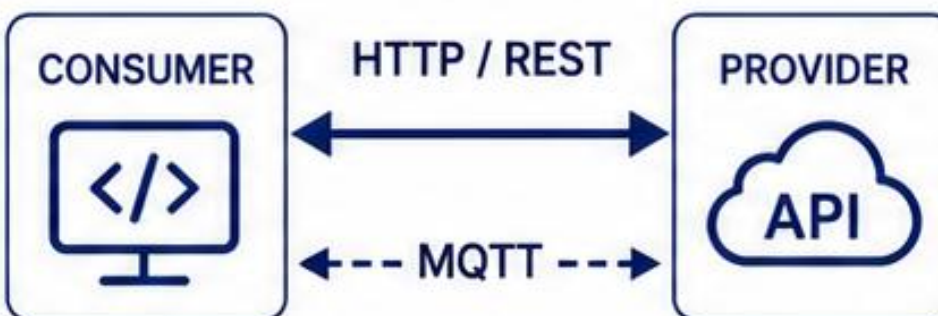
Thực hành tích hợp dịch vụ — chạy thật, gọi thật, có minh chứng thật

1 SERVICE CHẠY ĐƯỢC



- Docker Compose up
- Container healthy
- GET /health thành công

2 TÍCH HỢP THEO CONTRACT



- REST endpoint hoặc MQTT topic
- Payload đúng schema
- Consumer gọi được Provider

3 MINH CHỨNG RÕ RÀNG



- Log
- Screenshot
- Request/Response
- Payload MQTT

Buổi 6 không phải buổi thuyết trình slide.

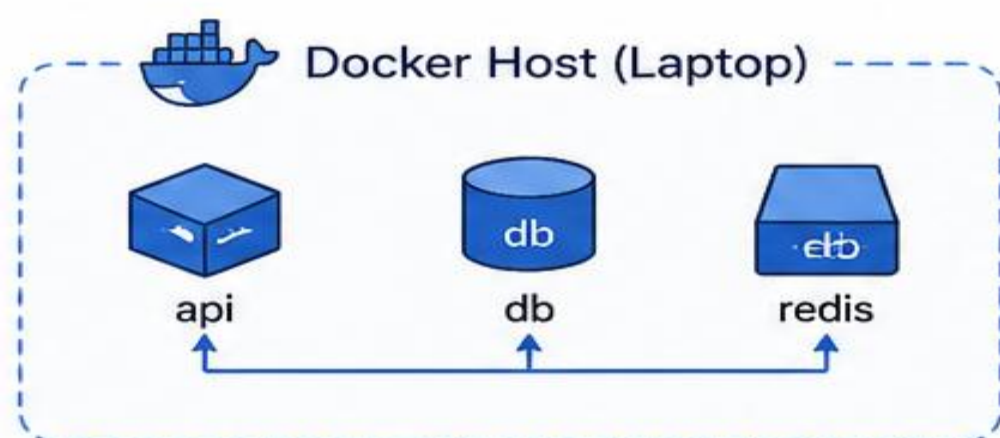
Trọng tâm là khả năng bắt tay thật giữa các service.



3 LOẠI MẠNG CẦN PHÂN BIỆT

Trong Buổi 6 sẽ có ba loại mạng làm việc song song. Cần phân biệt rõ để cấu hình đúng.

1 MẠNG DOCKER NỘI BỘ TRONG MỘT MÁY

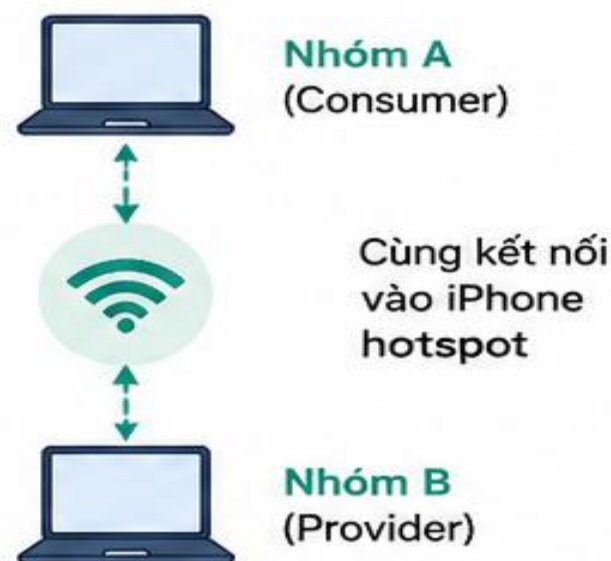


Các container trong cùng một docker-compose gọi nhau bằng **TÊN SERVICE**.

Ví dụ:

api gọi db: `http://db:5432`
api gọi redis: `http://redis:6379`

2 MẠNG LAN QUA HOTSPOT (GIỮA CÁC MÁY)



Gọi service của nhóm khác bằng **IP** của máy đối tác trong cùng hotspot.

Ví dụ:

`curl http://172.20.10.5:8000/health`

3 MẠNG INTERNET QUA HIVEMQ (NẾU DÙNG MQTT)



Các nhóm publish/subscribe topic qua HiveMQ Cloud trên **Internet**.

Ví dụ:

`mqtt://xxxxxx.hivemq.cloud:8883/smart-campus/events/...`

TỔNG HỢP

1	Container trong cùng máy	→	Tên service Docker	<code>http://api:8000</code>
2	Máy nhóm A → máy nhóm B (REST)	→	IP máy đối tác	<code>http://172.20.10.5:8000</code>
3	Nhóm A ↔ Nhóm B qua MQTT broker	→	Topic + broker URL	<code>mqtt://...hivemq.cloud:8883/topic</code>



Docker network như **team-internal** hoặc **class-net** chỉ hoạt động trong phạm vi một Docker host. Khi các nhóm chạy trên nhiều laptop, Docker service name **KHÔNG** dùng được để gọi chéo.



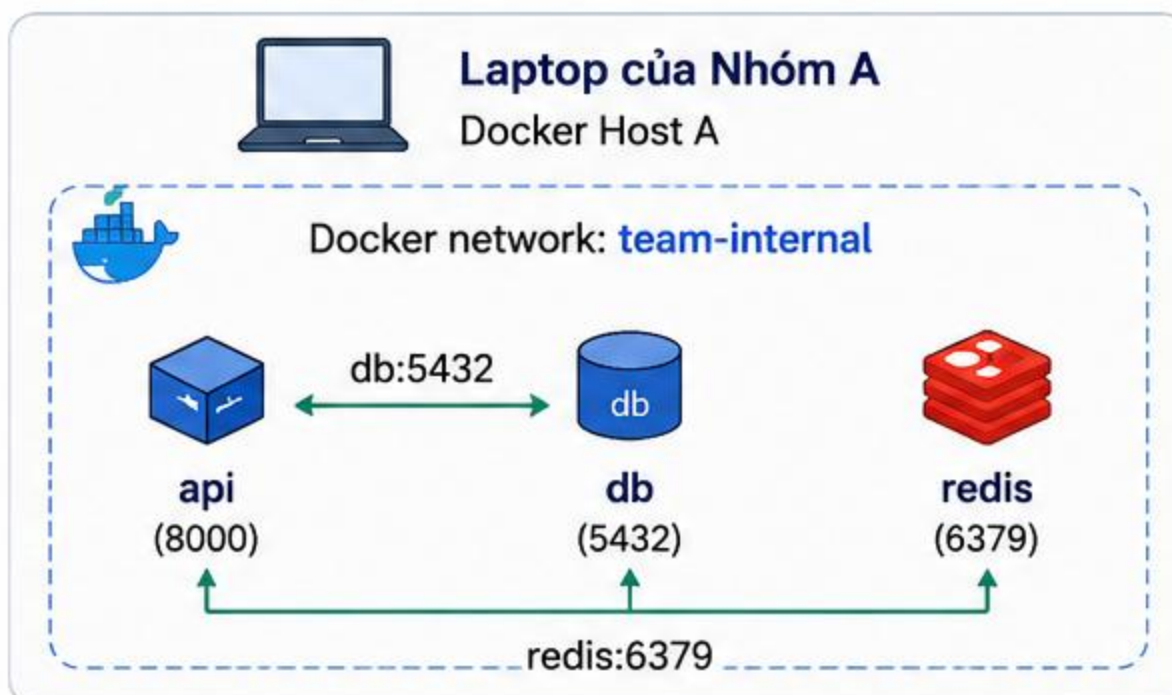
DOCKER SERVICE NAME CHỈ DÙNG TRONG CÙNG LAPTOP

Tên service (vd: db, api, ai-service) chỉ hoạt động trong mạng Docker của một Docker host.



ĐÚNG: TRONG CÙNG MỘT LAPTOP

Các container trong cùng một docker-compose gọi nhau bằng **TÊN SERVICE**.



Ví dụ trong code/container:



```
DB_HOST=db
REDIS_HOST=redis
AI_SERVICE_URL=http://ai-service:8000/predict
```

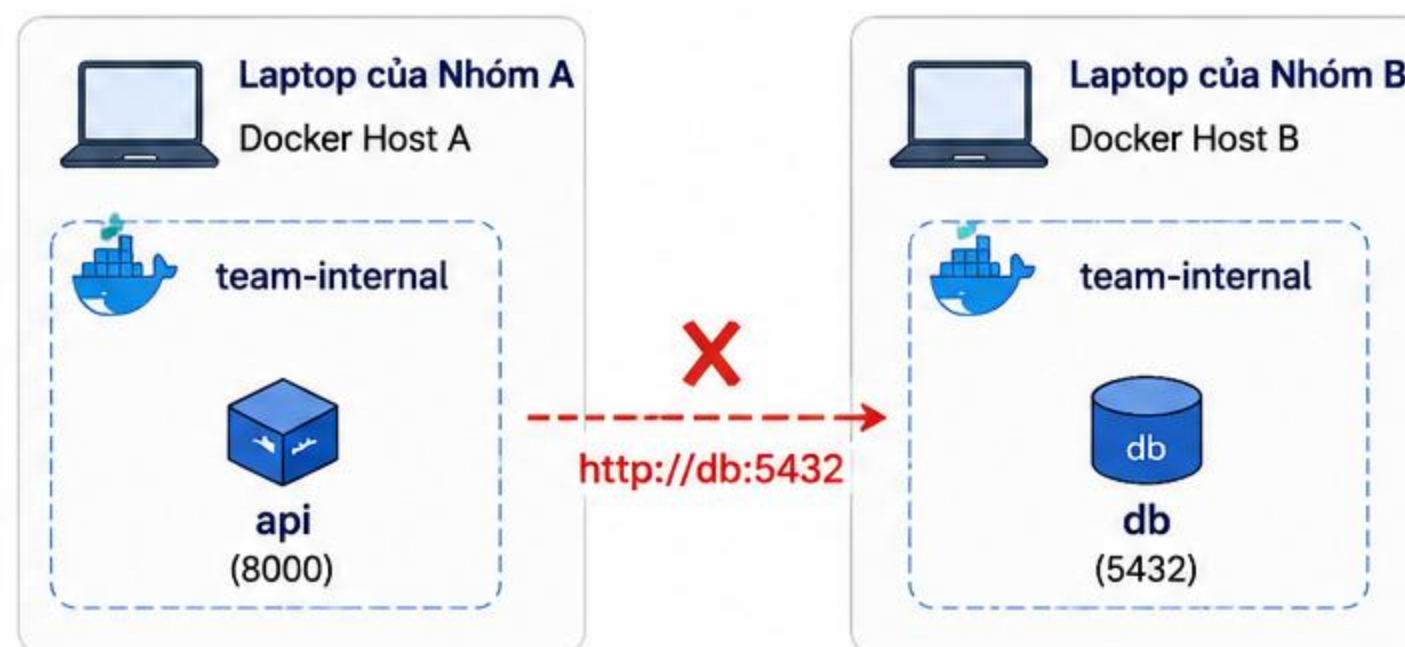


Chỉ dùng tên service trong phạm vi **CÙNG MỘT DOCKER HOST**.



SAI: QUA LAPTOP KHÁC

Không dùng tên service để gọi sang laptop khác.
Tên service **KHÔNG** resolve qua mạng LAN.



Ví dụ SAI:

```
DATABASE_URL=postgresql://user:pass@db:5432/app
```



Ví dụ ĐÚNG (qua LAN):

```
DATABASE_URL=postgresql://user:pass@172.20.10.5:5432/app
```



GHI NHỚ: Docker network (vd: team-internal, class-net) chỉ hoạt động trong phạm vi một Docker host.
Khi qua laptop khác, phải dùng **IP** của máy đối tác (vd: **172.20.10.x**) + port.



GỌI NHÓM KHÁC BẰNG IP QUA HOTSPOT

Khi gọi service của nhóm khác (máy khác), **KHÔNG** dùng tên service Docker.
Hãy gọi bằng **IP của máy đối tác** trong cùng hotspot.



LƯU Ý QUAN TRỌNG

- ✓ IP có thể thay đổi mỗi lần bật lại hotspot. Đầu Buổi 6 cần lấy IP lại và cập nhật `.env`.
- ✓ Đảm bảo cả hai máy đều đang kết nối cùng một hotspot.
- ✓ Không dùng `localhost`, `127.0.0.1` hoặc tên `service Docker` để gọi máy khác.



KIỂM TRA KẾT NỐI CƠ BẢN

- 1 Từ máy Nhóm A, ping tới IP của Nhóm B

```
$ ping 172.20.10.5
```

 ✓ OK
- 2 Gọi /health của Nhóm B

```
$ curl http://172.20.10.5:8000/health
```

 ✓ OK



MẸO

Mỗi nhóm nên ghi IP và port của mình vào bảng IP chung (mục 8.2) để các nhóm khác dễ dàng tích hợp.

LẤY IP CỦA MÁY ĐỐI TÁC TRONG HOTSPOT



IP có thể thay đổi mỗi lần bật lại hotspot.
Đầu Buổi 6 cần lấy lại IP và cập nhật vào `.env` của mình.

1 LẤY IP CỦA MÁY MÌNH

Kiểm tra IP của máy mình để xác nhận đã vào đúng hotspot (dải `172.20.10.x`).



Windows

Mở Command Prompt và chạy:

```
> ipconfig
```

Ví dụ kết quả:

```
Wireless LAN adapter Wi-Fi:  
IPv4 Address . . . . . : 172.20.10.6  
Subnet Mask . . . . . : 255.255.255.240  
Default Gateway . . . . : 172.20.10.1
```



macOS

Mở Terminal và chạy:

```
$ ipconfig getifaddr en0
```

Ví dụ kết quả:

```
172.20.10.6
```



Nếu IP không thuộc dải `172.20.10.0/28`
=> Chưa vào đúng hotspot iPhone.

2 LẤY IP CỦA MÁY ĐỐI TÁC

Hỏi nhóm bạn và kiểm tra IP của họ bằng một trong các cách sau:



Cách 1: Hỏi trực tiếp

Nhóm bạn chạy lệnh ở bên trái và đọc IP cho bạn.



Cách 2: Xem trong trang quản trị của iPhone hotspot

Settings → Personal Hotspot → Allow Others to Join → Wi-Fi Password → Xem danh sách thiết bị đã kết nối và IP của từng thiết bị.



Cách 3: Quét mạng (advanced)

Quét dải `172.20.10.0/28` để tìm các IP đang hoạt động.
Ví dụ dùng nmap trên Linux/macOS:

```
$ nmap -sn 172.20.10.0/28
```

3 CẬP NHẬT IP VÀO .ENV

Sau khi có IP của máy đối tác, cập nhật vào file `.env` (hoặc biến môi trường tương ứng).

Ví dụ: Nhóm A gọi API của Nhóm B



Nhóm A

HTTP



Nhóm B
IP: 172.20.10.5

`.env` (của Nhóm A)

```
# URL service của Nhóm B  
PARTNER_B_API_URL=http://172.20.10.5:8000
```



LƯU Ý

Mỗi lần bật lại hotspot hoặc máy đổi IP, hãy kiểm tra và cập nhật `.env` trước khi chạy.



CHECK
NHANH

✓ Máy mình có IP 172.20.10.x → ✓ Lấy đúng IP của máy đối tác → ✓ Cập nhật `.env` → ✓ Test kết nối

Ví dụ: 172.20.10.6

Ví dụ: 172.20.10.5

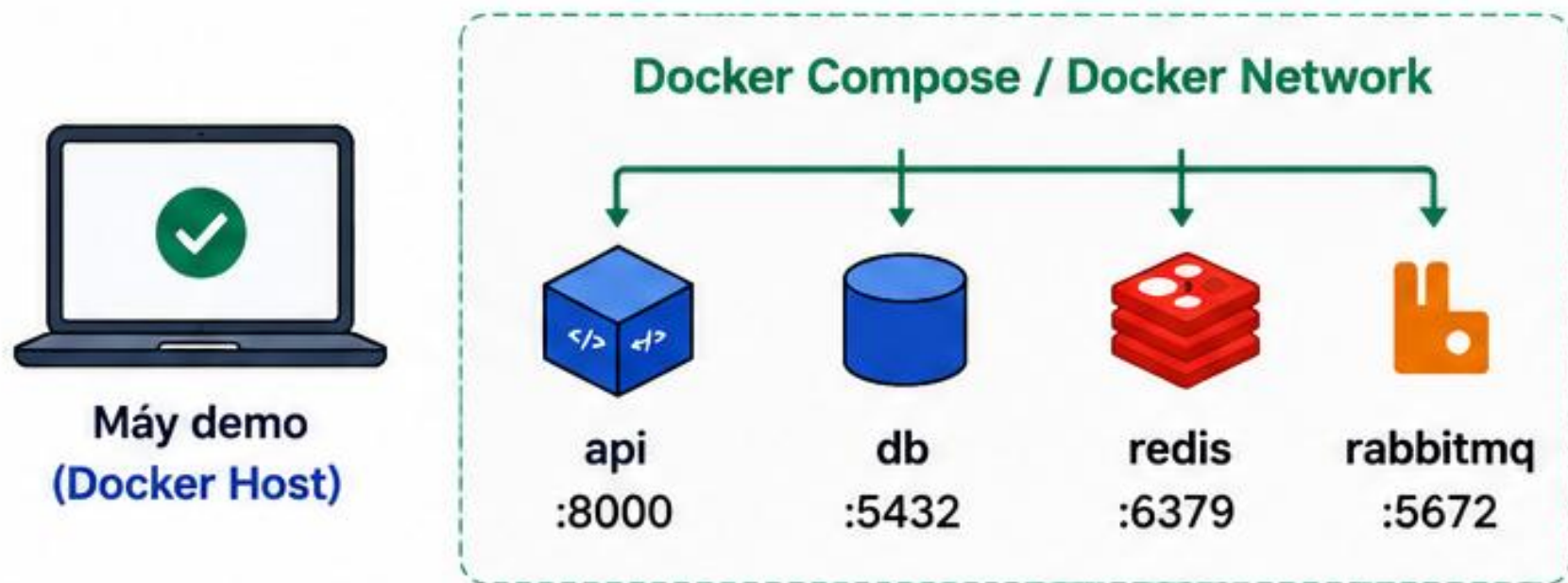
PARTNER_URL=...

curl http://172.20.10.5:8000/health

MỖI NHÓM CHỈ CHỌN MỘT MÁY DEMO ĐẠI DIỆN

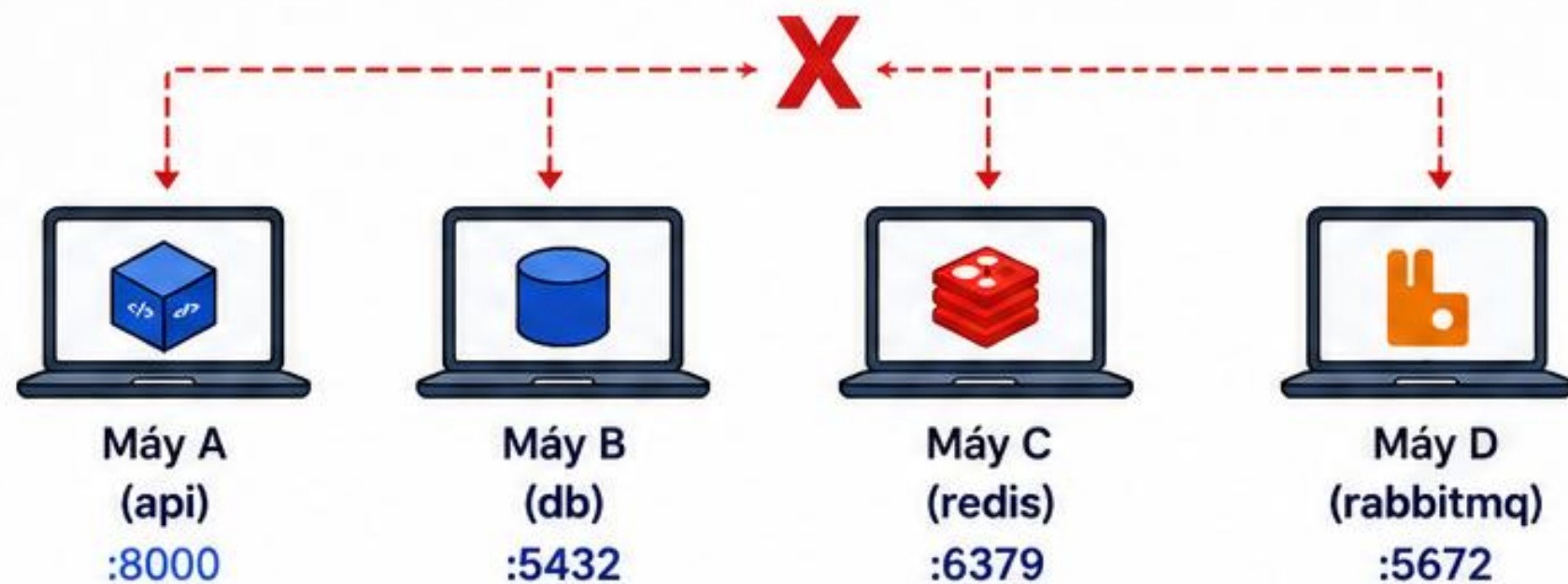
Toàn bộ stack (container) của nhóm chạy trên **CÙNG MỘT DOCKER HOST** (một laptop).

✓ ĐÚNG: CHẠY TRÊN MỘT MÁY DUY NHẤT



- ✓ Các container gọi nhau bằng **TÊN SERVICE**.
Docker network (vd: **team-internal**) hoạt động đúng.

✗ SAI: CHẠY TRÊN NHIỀU MÁY



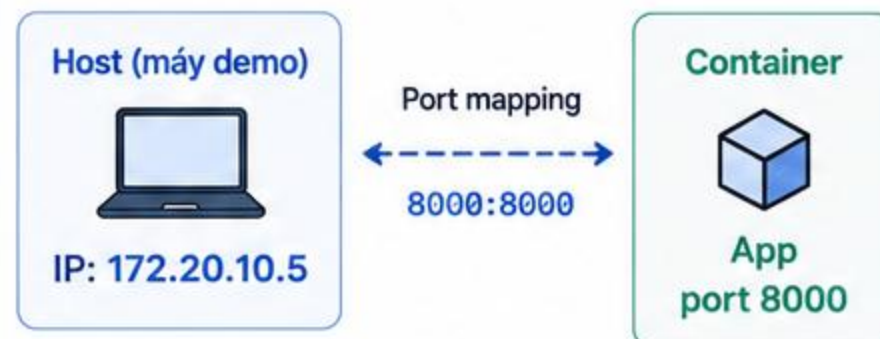
- ✗ Tên service **KHÔNG** resolve qua mạng LAN.
Docker network **KHÔNG** hoạt động giữa các laptop.

3 ĐIỀU ĐỂ NHÓM KHÁC GỌI ĐƯỢC REST API CỦA BẠN

Nếu thiếu 1 trong 3 điều sau, nhóm khác sẽ **KHÔNG** gọi được service của bạn qua IP.

1 MAPPING PORT RA HOST (docker-compose.yml)

Expose port container ra cổng trên máy host.



Ví dụ docker-compose.yml

```
services:
  api:
    image: my-api:latest
    ports:
      - "8000:8000" # host:container
    restart: unless-stopped
```

i Sau khi đổi port, chạy lại stack:

```
docker compose down && docker compose up -d
```

2 BIND 0.0.0.0 TRONG ỨNG DỤNG (không chỉ listen localhost)

Service phải lắng nghe trên **tất cả interface**.



Ví dụ:

Node.js / Express

```
app.listen(8000, '0.0.0.0');
```

Python / Flask

```
app.run(host='0.0.0.0', port=8000)
```

✓ **Kiểm tra nhanh trong container:**

```
netstat -tulnp | grep 8000
```

Kỳ vọng: 0.0.0.0:8000 hoặc :::8000

3 MỞ FIREWALL TRÊN MÁY HOST (Windows / macOS / Linux)

Cho phép **traffic vào port** của service.

Windows

```
# Mở port 8000 (TCP) trên Windows Firewall
netsh advfirewall firewall add rule name="API8000" ^
dir=in action=allow protocol=TCP localport=8000
```

macOS

```
# Mở port 8000
sudo pfctl -f /etc/pf.conf
```

i Hoặc tạm tắt firewall khi test (System Settings → Firewall)

Linux (UFW)

```
# Mở port 8000
sudo ufw allow 8000/tcp
```

i Kiểm tra: `sudo ufw status`



TỔNG KẾT

1

Mapping port
host:container

+

2

Ứng dụng listen
0.0.0.0

+

3

Firewall cho phép
port đó

→



Nhóm khác gọi được
`http://IP_HOST:PORT/health`

ENDPOINT CHUNG /HEALTH



MỌI NHÓM BẮT BUỘC EXPOSE ENDPOINT **/health**
Dùng để verify service đang sống và sẵn sàng.



GET /health → 200 OK

CÁCH HOẠT ĐỘNG

Client (nhóm khác)



HTTP GET
/health



200 OK



Service của bạn
(API container)



(Tùy chọn) Kiểm tra phụ thuộc



Database
(ví dụ: PostgreSQL)



Cache
(ví dụ: Redis)



Message Broker
(ví dụ: RabbitMQ)



Nếu mọi thứ OK
→ trả về **200 OK**



Nếu lỗi (DB down, timeout, ...)
→ trả về **5xx**

</> EXAMPLE RESPONSE

Success

```
{
  "status": "ok",
  "service": "ai-service",
  "version": "1.0.0",
  "uptime": 12345,
  "timestamp": "2025-05-20T10:15:30Z"
}
```

200 OK

Chỉ cần đảm bảo HTTP 200 là thành công.

🔍 CÁCH TEST

1. Trên máy của bạn (local)

```
curl http://localhost:8000/health
```

200 OK

2. Từ nhóm khác (qua IP hotspot)

```
curl http://172.20.10.5:8000/health
```

200 OK



Nếu /health trả về 200 OK → service đang sống.
Nếu không → nhóm khác sẽ không tích hợp được.



LƯU Ý



/health phải nhẹ, nhanh, không làm việc nặng.



Không cần trả quá nhiều thông tin,
nhưng đủ để biết dịch vụ đang LIVE.



Nên kiểm tra các phụ thuộc quan trọng
(DB, Redis, MQ....) để phản ánh đúng trạng thái.



Mọi service chính (API, worker...) đều nên có /health.



ĐIỂM CHUNG DUY NHẤT: Tài liệu này chỉ quy định /health là endpoint bắt buộc.

Các endpoint REST khác và topic MQTT do từng cặp nạp nệm ở nhóm tự thống nhất theo contract.



CẬP NHẬT .ENV – KHÔNG HARDCODE IP



IP hotspot có thể thay đổi mỗi lần bật lại.

Đầu Buổi 6 cần lấy IP lại và cập nhật vào file `.env` của nhóm.



DON'T: HARDCODE IP

Không viết cứng IP của nhóm đối tác trong code.

```
import requests
url = "http://172.20.10.5:8000/api/rooms"
response = requests.get(url)
```



Khi IP thay đổi (ví dụ: 172.20.10.8) → code sai, phải sửa code và build lại.



DO: SỬ DỤNG .ENV

Đưa thông tin nhóm đối tác vào file `.env`.

Code đọc từ biến môi trường.

1) File `.env`

```
# Partner services
PARTNER_A_API_URL=http://172.20.10.5:8000
PARTNER_A_TIMEOUT=5
MQTT_BROKER_URL=mqtts://a1b2c3d4.hivemq.cloud:8883
MQTT_USERNAME=team6
MQTT_PASSWORD=secret123
```

2) Code đọc từ `.env`

```
import os
import requests

base_url = os.getenv("PARTNER_A_API_URL")
url = f"{base_url}/api/rooms"
timeout = int(os.getenv("PARTNER_A_TIMEOUT", 5))
response = requests.get(url, timeout=timeout)
```



KHI IP THAY ĐỔI



1. Kết nối hotspot
và lấy IP mới
(vd: 172.20.10.8)



2. Cập nhật IP mới
vào file `.env`
`PARTNER_A_API_URL=...`



3. Restart Docker
Compose (nếu cần)
`docker compose up -d`



4. Test lại kết nối
/health hoặc
endpoint tích hợp



LỢI ÍCH

- ✓ Không cần sửa code khi IP đổi
- ✓ Dễ cấu hình theo từng môi trường (local / class / demo)
- ✓ An toàn khi push code lên Git
- ✓ Dễ quản lý, dễ bảo trì

MẪU .ENV THAM KHẢO



```
# ===== Partner Services (REST) =====
PARTNER_A_API_URL=http://172.20.10.5:8000
PARTNER_B_API_URL=http://172.20.10.6:8000
REQUEST_TIMEOUT=5
```

```
# ===== MQTT (nếu dùng) =====
MQTT_BROKER_URL=mqtts://a1b2c3d4.hivemq.cloud:8883
MQTT_USERNAME=team6
MQTT_PASSWORD=secret123
```

```
# ===== App Config =====
APP_PORT=8000
LOG_LEVEL=INFO
ENV=demo
```


XÁC MINH ENDPOINT /HEALTH CỦA NHÓM KHÁC



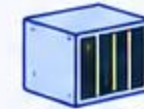
Khi gọi API của nhóm khác, hãy kiểm tra endpoint `/health` trước để đảm bảo dịch vụ của họ đang hoạt động (200 OK).



Nhóm A



/health



Nhóm B

1 QUY TRÌNH XÁC MINH



1. Gọi `/health` của nhóm khác

GET `http://IP:PORT/health`



2. Kiểm tra trạng thái

200 OK = Live
Khác = Không dùng



3. Nếu OK mới gọi API khác

GET `http://IP:PORT/api/...`
POST `http://IP:PORT/api/...`



4. Ghi log và xử lý lỗi

Timeout, 5xx, Network
→ Không gọi tiếp

2 CÁCH GỌI /HEALTH



Python (requests)

```
import requests
url = "http://172.20.10.5:8000/health"
resp = requests.get(url, timeout=3)
if resp.status_code == 200:
    print("Service is LIVE")
else:
    print(f"Service is NOT OK, code={resp.status_code}")
```

GIẢI THÍCH

- timeout=3: chờ tối đa 3 giây
- 200 OK: dịch vụ sẵn sàng



Node.js (axios)

```
import axios from 'axios';
try {
    const res = await axios.get('http://172.20.10.5:8000/health', {
        timeout: 3000
    });
    if (res.status === 200) console.log('Service is LIVE');
    else console.log('Service is NOT OK, code=' + res.status);
} catch (err) {
    console.log('Cannot reach /health:', err.message);
}
```

GIẢI THÍCH

- timeout=3000ms
- Có try/catch để bắt lỗi mạng

3 KỊCH BẢN PHẢN HỒI

Dạng phản hồi	Ý nghĩa	Xử lý
200 OK	Dịch vụ hoạt động tốt	Tiếp tục gọi các API khác
3xx (Redirect)	Chuyển hướng	Có thể theo redirect (nếu cần)
4xx (VD: 404, 401)	Client error (sai URL, không quyền, ...)	Không gọi tiếp, báo lỗi cho nhóm bạn
5xx (VD: 500, 503)	Server error	Không gọi tiếp, thử lại sau (backoff / retry policy)
Timeout / Network Error	Không kết nối được	Không gọi tiếp, thông báo lỗi mạng

4 MẪU LOG THAM KHẢO

```
[2025-05-20 10:15:30] INFO Calling health: http://172.20.10.5:8000/health
[2025-05-20 10:15:30] INFO Health check result: 200 OK (120ms)
[2025-05-20 10:15:30] INFO Service is LIVE, continue calling APIs
[2025-05-20 10:15:31] ERROR Health check failed: timeout after 3000ms
```

5 LƯU Ý

- ✓ Luôn kiểm tra `/health` trước khi gọi API của nhóm khác. Đừng giả định dịch vụ luôn sẵn sàng.
- ✓ Đặt timeout hợp lý (2-5 giây) để tránh treo chương trình.
- ✓ Ghi log đầy đủ để dễ dàng debug và theo dõi.
- ✓ Endpoint `/health` nên nhẹ, nhanh và không phụ thuộc dịch vụ bên ngoài.



LỊCH HỌC 5 TIẾT – KẾ HOẠCH LÀM & DEMO



Mục tiêu: Sinh viên làm việc theo nhóm, xây dựng hệ thống và demo sản phẩm thực tế.



3 tiết đầu: Chuẩn bị



2 tiết cuối: Chấm demo

TIẾT 1 CHUẨN BỊ



- Giới thiệu đề bài & yêu cầu
- Hướng dẫn công nghệ và kiến trúc hệ thống
- Phân công nhóm, vai trò
- Lập kế hoạch & phân tích yêu cầu

KẾT QUẢ

Kế hoạch chi tiết của nhóm

TIẾT 2 CHUẨN BỊ



- Thiết kế hệ thống chi tiết
- Xây dựng môi trường dev (repo, Docker, .env)
- Xây dựng cấu trúc dự án
- Phân chia task cho từng thành viên

KẾT QUẢ

Thiết kế hệ thống và môi trường dev sẵn sàng

TIẾT 3 CHUẨN BỊ



- Phát triển các service (API, DB, Cache, ...)
- Tích hợp nội bộ
- Viết test & kiểm thử cơ bản
- Chuẩn bị dữ liệu demo

KẾT QUẢ

Hệ thống chạy được trên môi trường dev

TIẾT 4 CHẤM DEMO



- Nhóm demo sản phẩm (10–15 phút/nhóm)
- Giảng viên đặt câu hỏi
- Đánh giá theo rubric
- Góp ý và hướng dẫn cải thiện

KẾT QUẢ

Điểm demo + Nhận xét chi tiết

TIẾT 5 CHẤM DEMO



- Hoàn thiện & demo bổ sung (nếu cần)
- Giảng viên đánh giá cuối cùng
- Công bố kết quả
- Tổng kết khóa học

KẾT QUẢ

Điểm cuối cùng + Nhận xét tổng kết



LƯU Ý CHUNG

- ✓ 3 tiết đầu (Chuẩn bị): tập trung xây dựng & hoàn thiện hệ thống.
- ✓ 2 tiết cuối (Chấm demo): trình bày sản phẩm và nhận đánh giá.
- ✓ Mỗi nhóm nộp báo cáo + link repo + hướng dẫn chạy hệ thống.



LÀM VIỆC NHÓM

- Tôn trọng tiến độ chung.
- Phân công rõ ràng.
- Hỗ trợ lẫn nhau để đạt kết quả tốt nhất.

Thank You!

DAI NAM
UNIVERSITY